

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №5

Выполнил:

Попов Никита

К3344

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- подключить и настроить rabbitMQ/kafka;
- реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Ход работы

Будем запускать kafka в докере: настроим docker compose.

```
kafka:
  image: public.ecr.aws/bitnami/kafka:latest
  ports:
    - "9092:9092"
  environment:
    - KAFKA_CFG_NODE_ID=1
    - KAFKA_CFG_PROCESS_ROLES=broker,controller
    - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
    - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
    - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092
    - KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
    - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
    - ALLOW_PLAINTEXT_LISTENER=yes
  healthcheck:
    test: ["CMD-SHELL", "kafka-topics.sh --bootstrap-server localhost:9092 --list || exit 1"]
    interval: 10s
    timeout: 10s
    retries: 10
```

Также настроим kafdrop (listener-ui), с помощью которого можно просматривать сообщения:

```
kafdrop:
  image: obsidiandynamics/kafdrop
  ports:
    - "9000:9000"
  environment:
    KAFKA_BROKERCONNECT: kafka:9092
  depends_on:
    kafka:
      condition: service_healthy
```

Настроим взаимодействие между сервисом отзывов и ресторанов:

code > services > reviews-service > src > ts > kafka.ts > ...

```
import { Kafka } from "kafkajs";
import { config } from "../config";

const kafka = new Kafka({ clientId: "reviews-service", brokers: config.kafkaBrokers });
const producer = kafka.producer();
let connected = false;

export async function publish(topic: string, payload: object) {
  if (!connected) {
    await producer.connect();
    connected = true;
  }
  await producer.send({ topic, messages: [{ value: JSON.stringify(payload) }] });
}
```

```
const consumerRetryMs = 3000;
const reviewTopics = ["reviews.review.created", "reviews.review.updated", "reviews.review.deleted"];

export async function publish(topic: string, payload: object) {
  if (!connected) {
    await producer.connect();
    connected = true;
  }
  await producer.send({ topic, messages: [{ value: JSON.stringify(payload) }] });
}

type ReviewEventPayload = {
  restaurantId?: string;
};
```

```

export function startReviewEventsConsumer(onReviewChanged: (restaurantId: string) => Promise<void>) {
  if (consumerStarted) return;
  consumerStarted = true;

  const run = async () => {
    while (true) {
      try {
        await admin.connect();
        await admin.createTopics({
          waitForLeaders: true,
          topics: reviewTopics.map((topic) => ({ topic, numPartitions: 1, replicationFactor: 1 })))
        });
        await admin.disconnect();

        await reviewEventsConsumer.connect();
        for (const topic of reviewTopics) {
          await reviewEventsConsumer.subscribe({ topic, fromBeginning: false });
        }

        console.log("KAFKA_BROKERS =", process.env.KAFKA_BROKERS);
        console.log("config.kafkaBrokers =", config.kafkaBrokers);

        await reviewEventsConsumer.run({
          eachMessage: async ({ topic, message }) => {
            if (!message.value) return;

            try {
              const payload = JSON.parse(message.value.toString()) as ReviewEventPayload;
              if (!payload.restaurantId) {
                console.warn(`[kafka] skip ${topic}: restaurantId is missing`);
                return;
              }

              await onReviewChanged(payload.restaurantId);
              console.log(`[kafka] handled ${topic} for restaurant ${payload.restaurantId}`);
            } catch (error) {
              console.error(`[kafka] failed to handle ${topic}`, error);
            }
          }
        });
      }
    }
  });
}

```

В итоге при создании отзыва отправляется сообщение:

Topic Messages: reviews.review.created

Partition: 0

Offset: 1

Key: empty

Timestamp: 2026-05-06 07:07:08.553

Headers: empty



```
{
  "userId": "afedcc7a-06e3-41e0-b869-02921c19de67",
  "restaurantId": "66c211f2-a013-4ae4-8a03-46cc88e354ac",
  "rating": 5,
  "text": "овцовой",
  "id": "ee6559ce-ff03-4572-ae56-f16fd800126b"
}
```

Вывод

В итоге подключили и настроили Kafka для взаимодействия сервисов.